설비 고장을 예측하는 AI 데이터 분석

문자열 다루기

공백과 대소문자 정리

posco

설비 고장을 예측하는 AI 데이터 분석

문자열 다루기

문자열의 이해와 생성 Part 1

문자열 만들기

자료형과 형변환

인덱싱과 슬라이싱 Part 2

인덱싱

슬라이싱

연산·검색과 정리 메서드 Part 3

정리 메서드

f-string과 텍스트 정리 Part 4

f-string

공백과 대소문자 정리

posco

잠깐 — 값은 '객체', 메서드는 객체의 기능

값 하나하나가 '객체' = 값 + 그 값에 딸린 기능

자료형은 객체의 종류 → 문자열엔 문자열 기능, 숫자엔 숫자 기능

오늘 배울 '메서드'는 이 값(객체)에 딸린 기능을 점(.)으로 부르는 것

(참고 - 변수는 메모리의 값에 붙인 이름표, =는 오른쪽을 먼저 계산해 저장)

메서드란 무엇인가

메서드 - 특정 자료형(값=객체)에 소속된 함수

startswith·count처럼 값 뒤에 점을 찍어 부름

문자열마다 딸린 정리·검색 기능 (대문자 바꾸기·공백 떼기 등)

지금은 문자열의 메서드부터 익힘

메서드 사용법 - 점(.) 문법

'문자열.메서드이름()' 형태로 호출

괄호를 꼭 붙여야 실행됨 (word.upper)

추가 정보가 필요하면 괄호 안에 인자 (count('a'))

인자 없으면 괄호만 비워 둠

점 문법 - 코드 예시

문자열.메서드이름() 형태로 호출

CODE

```
word = 'python'
print(word.upper())          # PYTHON
print(word.count('p'))       # 1
print(word.startswith('p'))  # True
```

원본은 바뀌지 않는다

문자열 메서드는 **원본을 안 바꾸고 새 결과**를 돌려줌

word.upper()를 해도 word 자체는 그대로

결과를 쓰려면 반드시 변수에 다시 받기

word = word.upper() 패턴

upper() - 모두 대문자로

upper() - 영문 글자를 모두 대문자로

'python'.upper()는 'PYTHON'

한글·숫자·기호는 그대로 ('정상 normal'은 '정상 NORMAL')

결과는 변수에 다시 받기

실습 1. 대문자로 바꾸기

목표	upper() 로 문자열을 모두 대문자로 변환
단계	① 'ready' 저장 → ② upper() 결과를 새 변수에 받아 출력
예상 결과	READY

실습 1. 대문자로 바꾸기 정답

CODE

```
s = "ready"
big = s.upper()
print(big)      # READY
```

lower() - 모두 소문자로

lower() - 영문 글자를 모두 소문자로

'NORMAL'.lower()는 'normal'

한글·숫자·기호는 그대로

실무에서는 lower를 더 자주 사용

실습 2. 소문자로 바꾸기

목표	lower() 로 문자열을 모두 소문자로 변환
단계	① 'WARNING' 저장 → ② lower() 결과를 새 변수에 받아 출력
예상 결과	warning

실습 2. 소문자로 바꾸기 정답

CODE

```
s = "WARNING"
small = s.lower()
print(small)    # warning
```

대소문자 통일이 필요한 이유

같은 뜻인데 표기만 다른 데이터를 **하나로 통일**

'NORMAL' 'Normal' 'normal'이 한 파일에 섞임

통일 안 하면 같은 상태가 따로 집계

보통 lower로 모두 소문자로 맞춤

capitalize()·title()

첫 글자·단어 첫 글자만 대문자로 만드는 메서드

capitalize - 문장 첫 글자만 ('python is'-'Python is')

title - 단어마다 첫 글자 ('hong gil'-'Hong Gil')

이름·제목 정리에 가끔 사용

실습 3. 단어 첫 글자 대문자로 만들기

목표	title() 로 각 단어의 첫 글자를 대문자로 표준화
단계	① 'kim chul soo' 저장 → ② title() 로 변환해 출력
예상 결과	Kim Chul Soo

실습 3. 단어 첫 글자 대문자로 만들기 정답

CODE

```
name = "kim chul soo"
print(name.title())    # Kim Chul Soo
```

대소문자 무시 비교 - lower() 활용

비교 전 양쪽을 **lower로 맞추면** 대소문자와 무관하게 비교

'NORMAL'.lower() == 'normal'.lower()는 True

검색도 대상과 단어를 lower로 맞춰 탐색

데이터를 다룰 때 표준처럼 쓰는 패턴

실습 4. 대소문자 무시하고 비교하기

목표	lower() 로 통일한 뒤 == 로 대소문자 무시 비교
단계	① 'Fault'·'FAULT' 저장, 그대로 == 비교 → ② 양쪽 lower() 후 == 비교
예상 결과	False → True

실습 4. 대소문자 무시하고 비교하기 정답

CODE

```
a = "Fault"
b = "FAULT"
print(a == b)                  # False (대소문자 달라 다른 값)
print(a.lower() == b.lower())  # True  (소문자로 통일 후 비교)
```

isupper()·islower() 검사

Q1. isupper()·islower()는 무엇인가요?

A. 모두 대문자/소문자인지 참·거짓으로 확인

Q2. 결과는?

A. True 또는 False (바꾸는 게 아닌 확인)

Q3. is로 시작하는 메서드는?

A. 대체로 ~인지 확인하는 검사용

실습 5. 대문자인지 소문자인지 검사하기

목표	isupper()·islower() 로 대문자/소문자 여부 검사
단계	① 'ABC'·'abc'·'Abc' → ② isupper()·islower() 로 각각 검사
예상 결과	True / True / False

실습 5. 대문자인지 소문자인지 검사하기 정답

CODE

```
print("ABC".isupper())   # True
print("abc".islower())   # True
print("Abc".isupper())   # False
```

실습 6. 파일명 규칙 한 번에 점검하기

목표	lower() 로 통일한 뒤 startswith·endswith 로 파일명 규칙을 한 번에 점검
단계	① 'Sensor_LOG.CSV ' 라는 값을 변수에 저장 ② 소문자로 통일하여 sensor로 시작하는지와 .csv로 끝나는지 각각 확인
예상 결과	True / True (통일 전에는 False)

실습 6. 파일명 규칙 한 번에 점검하기 정답

CODE

```
fname = "Sensor_LOG.CSV"
low = fname.lower()                 # sensor_log.csv
print(low.startswith("sensor"))     # True
print(low.endswith(".csv"))         # True
print(fname.endswith(".csv"))       # False  (대문자 .CSV라 통일 전엔 False)
```

공백이 데이터를 망친다

눈에 안 보이는 **앞뒤 공백**이 데이터를 다른 값으로

'정상'과 '정상 '(뒤 공백)은 다른 값

같은 상태가 공백 차이로 따로 집계

다루기 전 앞뒤 공백 제거가 거의 필수

strip() - 앞뒤 공백 제거

strip() - 문자열의 앞뒤 공백을 떼어 줌

' 정상 '.strip()은 '정상'

원본은 그대로 - text = text.strip()으로 다시 받기

결과를 안 받으면 공백이 그대로 남는 실수 빈번

strip - 코드 예시

양옆에 기호를 붙여 공백 제거를 눈으로 확인

CODE

```
text = '  정상  '
print('[' + text + ']')      # [  정상  ]
text = text.strip()
print('[' + text + ']')      # [정상]
```

실습 7. 앞뒤 공백 제거하기

목표	strip() 로 문자열 앞뒤의 공백을 제거
단계	① ' 가동중 ' 저장 → ② strip() 결과를 양옆 기호 붙여 출력
예상 결과	[가동중] (앞뒤 공백 사라짐)

실습 7. 앞뒤 공백 제거하기 정답

CODE

```
s = "  가동중  "
clean = s.strip()
print("[" + clean + "]")    # [가동중]  (앞뒤 공백 제거)
```

lstrip()·rstrip()

lstrip은 왼쪽, rstrip은 오른쪽만 공백 제거

l은 left(앞), r은 right(뒤)

보통은 strip 하나로 양쪽 다 떼면 충분

한쪽 공백을 살려야 할 때만 사용

실습 8. 한쪽 공백만 제거하기

목표	lstrip()·rstrip() 로 한쪽 공백만 제거
단계	① ' 대기 ' 저장 → ② lstrip()(왼쪽)·rstrip()(오른쪽) 각각 적용
예상 결과	[대기] / [대기]

실습 8. 한쪽 공백만 제거하기 정답

CODE

```
s = "  대기  "
print("[" + s.lstrip() + "]")   # [대기  ]  (왼쪽만 제거)
print("[" + s.rstrip() + "]")   # [  대기]  (오른쪽만 제거)
```

strip(문자) - 특정 문자 제거

괄호에 글자를 넣으면 **양 끝**에서 그 글자 제거

'###정상###'.strip('#')은 '정상'

나올 때까지 양 끝에서 계속 제거

주의 - 가운데 글자는 건드리지 않음

실습 9. 특정 문자 제거하기

목표	strip('문자') 로 앞뒤의 특정 문자를 제거
단계	① '***경고***' 저장 → ② 앞뒤 * 제거
예상 결과	경고

실습 9. 특정 문자 제거하기 정답

CODE

```
s = "***경고***"
print(s.strip("*"))    # 경고  (앞뒤 * 제거)
```

strip으로 못 지우는 중간 공백

> strip은 **앞뒤만** - 글자 사이 공백은 못 지움

' 정 상 '.strip()은 '정 상' (가운데 공백 남음)

중간 공백은 다음 모듈

replace로 해결

strip은 앞뒤 전용임을 기억

공백·대소문자 정리 활용 - 입력값 정리

입력값은 보통 **strip 후 lower** 순으로 정리

' NORMAL '이 'normal'로 깔끔하게

정리하면 비교·집계가 정확해짐

입력값이 들어오면 일단 정리부터 하는 습관

메서드 연결해서 쓰기 (체이닝)

메서드 뒤에 또 메서드를 점으로 **이어 붙이기**

text.strip().lower()는 공백 제거 후 소문자

읽는 순서는 왼쪽에서 오른쪽으로

정리 단계가 여러 개일 때 코드가 깔끔

체이닝 - 코드 예시

strip().lower()로 두 단계를 한 줄에

CODE

```
raw = '  NORMAL  '
step1 = raw.strip()          # 'NORMAL'
clean = raw.strip().lower()  # 'normal'
print(clean)
```

실습 10. 메서드 이어 붙이기

목표	strip().lower() 를 한 줄로 이어 붙여 한 번에 정리
단계	① ' WARNING ' 저장 → ② strip().lower() 체이닝으로 정리해 출력
예상 결과	warning

실습 10. 메서드 이어 붙이기 정답

CODE

```
s = "  WARNING  "
print(s.strip().lower())   # warning  (공백 제거 후 소문자, 한 줄)
```

메서드 결과를 변수에 다시 저장

메서드 결과는 **변수에 다시 받아야** 실제로 쓸 수 있음

clean = text.strip().lower() 또는 같은 변수에 덮어쓰기

name = name.strip()이 가장 흔한 패턴

등호 오른쪽이 먼저 계산되어 왼쪽에 들어감

흔한 실수 - strip 결과 저장 안 함

Q1. text.strip()만 쓰고 끝내면?

A. 정리된 결과가 사라지고 원본 그대로

Q2. 어떻게 고치나요?

A. text = text.strip()으로 결과를 다시 받기

Q3. 값이 안 바뀐 것 같으면?

A. 결과를 다시 받았는지 먼저 확인

실습 11. 결과를 변수에 다시 저장하기

목표 메서드 결과를 변수에 다시 받아야 정리가 반영됨을 확인

단계 ① strip() 결과 안 받고 출력(그대로) → ② strip() 결과를 변수에 재할당한 뒤 출력(반영)

예상 결과 [경보] → [경보]

실습 11. 결과를 변수에 다시 저장하기 정답

CODE

```
s = "  경보  "
s.strip()                # 결과를 안 받으면 원본은 그대로
print("[" + s + "]")     # [  경보  ] (변화 없음)
s = s.strip()            # 다시 받아야 반영
print("[" + s + "]")     # [경보]
```

공백·대소문자 메서드 정리

원본은 안 바뀜, 결과를 다시 받기 (공통 원칙)

대소문자 - upper, lower, capitalize, title, isupper, islower

공백 - strip, lstrip, rstrip, strip(문자)

점으로 이어 붙이는 체이닝 가능

정리 메서드 비교표

메서드	기능
upper / lower	모두 대문자 / 소문자
capitalize / title	첫 글자 / 단어마다 첫 글자 대문자
strip	앞뒤 공백 제거
lstrip / rstrip / strip(문자)	한쪽·특정 문자 제거

감사합니다.

posco